

copia

DSL Reference & Interface Guide

Overview

copia is a MySQL database seeder driven by its own declarative mini-language, also called **copia**. You describe what each column should contain, pick a row count, and let the tool handle the rest – no boilerplate scripts, no manual data entry.

Interface

Database Tree (left panel)

Lists all tables in the connected database. Click a table to select it as the target for a Submit.

Copia DSL Editor (center)

Where you write your copia file. Typing `:` after a column name triggers an autocomplete dropdown listing available generators.

Action Bar (bottom)

Control	Role
<code>ROWS</code>	Number of rows to generate.
<code>RUN</code>	Parse, validate and preview – nothing is written to the DB.
<code>SUBMIT</code>	Same as RUN, then writes to the selected table.

Output Panel (bottom)

Shows parser output and per-generator validation results in real time.

```
parsing your input...
Column(name='nom', generator=GeneratorCall(name='name', ...))
Column(name='age', generator=GeneratorCall(name='int_range', ...))
Looks good
Validating your input...
Validating generator 'name'      → Valid!
Validating generator 'int_range' → Valid!
```

The copia DSL

Syntax

Each line defines one column:

```
column_name:generator(args)
```

Columns are separated by any whitespace – newlines, tabs, spaces. Order only matters when using **ref**.

```
nom:name()
email:email()
age:int_range(18, 60)
pays:enum("CI", "SN", "ML", "GH")
```

Optional parentheses

For zero-argument generators, parentheses can be omitted:

```
nom:name      # same as nom:name()
```

Positional arguments

```
age:int_range(18, 60)
score:float_range(0.0, 1.0)
pays:enum("CI", "SN", "ML")
```

Foreign keys – ref

ref pulls a value from already-inserted rows in another table. Pass the target as a "table.column" string:

```
user_id:ref("users.id")
```

Because ref reads live data, columns using it should come after the rows they reference have been submitted.

Runs vs Submits

Action	Effect
RUN	Parse, validate, generate data – DB untouched. A table does not need to be selected.
SUBMIT	Same as RUN, then inserts into the selected table.

Generator Catalogue

Identity

Generator	Optional parameters	Example
name	gender: 'male' 'female' 'nonbinary' 'all' = 'all'	nom:name

Generator	Optional parameters	Example
<code>firstname</code>	gender: 'male' 'female' 'nonbinary' 'all' = 'all'	<code>prenom:firstname</code>
<code>lastname</code>	gender: 'male' 'female' 'nonbinary' 'all' = 'all'	<code>famille:lastname</code>
<code>username</code>	—	<code>pseudo:username</code>
<code>password</code>	length=12, special_chars=True, upper_case=False, lower_case=False	<code>mdp:password(16, True)</code>
<code>phone</code>	—	<code>tel:phone</code>

Contact & Location

Generator	Optional parameters	Example
<code>email</code>	safe: bool = True, domain: str = ''	<code>email:email(False, "gmail.com")</code>
<code>address</code>	—	<code>adresse:address</code>
<code>city</code>	—	<code>ville:city</code>
<code>country</code>	—	<code>pays:country</code>
<code>zipcode</code>	—	<code>cp:zipcode</code>

Network

Generator	Optional parameters	Example
<code>ipv4</code>	network=False, address_class='c', private=False	<code>ip:ipv4(False, "b")</code>
<code>ipv6</code>	network: bool = False	<code>ip6:ipv6</code>
<code>url</code>	—	<code>site:url</code>
<code>user_agent</code>	—	<code>ua:user_agent</code>

Numbers

Generator	Optional parameters	Example
<code>int_range</code>	min: int = 0, max: int = 100	<code>age:int_range(18, 60)</code>
<code>float_range</code>	min: float = 0.0, max: float = 100.0	<code>score:float_range(0.0, 1.0)</code>

Choice & References

Generator	Optional parameters	Example
<code>enum</code>	<code>*args</code> (possible values)	<code>statut:enum("actif", "inactif")</code>
<code>ref</code>	<code>column: str – format "table.column"</code>	<code>user_id:ref("users.id")</code>
<code>uuid</code>	–	<code>id:uuid</code>

Text

Generator	Optional parameters	Example
<code>word</code>	–	<code>tag:word</code>
<code>sentence</code>	<code>length: int = 10, vary_length: bool = True</code>	<code>titre:sentence(6)</code>
<code>paragraph</code>	<code>length: int = 5, vary_length: bool = True</code>	<code>bio:paragraph(3, False)</code>

Dates

Generator	Optional parameters	Example
<code>date_of_birth</code>	–	<code>naissance:date_of_birth</code>
<code>past_date</code>	–	<code>created_at:past_date</code>
<code>future_date</code>	–	<code>expires_at:future_date</code>